

```

1  unit Form_Main;
2  {
3    Created 05/10/2023 By Roger Williams
4
5    "clone" of the SafeScan RA software
6
7    - does not communicate directly with the TA machines (boo!)
8    - can import attendance records (1_attlog.dat)
9    - can import full backup (zkdb.db) for extracting users
10   - uses SQLite as the TA8015 machines use that natively
11   - uses a central database (main.db) like SafeScan to store all users and
12     attendance data
13
14   Its Proposed Use:
15
16   - weekly attendance backups taken from the TA8015s and imported
17   - users maintained ON the TA8015s as per the manual
18   - if user added, get full backup from the TA8015 restore into the software
19     to update users list. (applies discipline to keep machines data maintained)
20   - departments are software only (as with SafeScan) and used for reporting
21   - have all the users on each TA8015
22
23   Creates main.db using SQLite as this is the same as the clocking machines use
24   Creates the tables from the text file:
25
26   schema.txt
27
28 }
29
30 interface
31
32 uses
33   Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants, System.Classes, Vcl.
34   Graphics,
35   Vcl.Controls, Vcl.Forms, Vcl.Dialogs, Data.FMTBcd, Vcl.StdCtrls, Data.DB,
36   Data.SqlExpr, Data.DbxSqlite, Datasnap.DBClient, Datasnap.Provider, Vcl.Grids,
37   Vcl.DBGrids, FireDAC.Stan.ExprFuncs, FireDAC.Phys.SQLiteWrapper.Stat,
38   FireDAC.Phys.SQLiteDef, FireDAC.Stan.Intf, FireDAC.Phys, FireDAC.Phys.SQLite,
39   FireDAC.Stan.Option, FireDAC.Stan.Error, FireDAC.UI.Intf, FireDAC.Phys.Intf,
40   FireDAC.Stan.Def, FireDAC.Stan.Pool, FireDAC.Stan.Async, FireDAC.VCLUI.Wait,
41   FireDAC.Comp.Client, FireDAC.Stan.Param, FireDAC.DatS, FireDAC.DApt.Intf,
42   FireDAC.DApt, FireDAC.Comp.DataSet, Vcl.Menus, Vcl.Buttons,
43   Vcl.Imaging.pngimage, Vcl.ExtCtrls, System.ImageList, Vcl.ImgList,
44   Vcl.ButtonGroup, Vcl.ComCtrls, Vcl.ToolWin, FRA_Reports;
45
46 type
47   TFRMMain = class(TForm)
48     Button1: TButton;
49     DBGrid1: TDBGrid;
50     FDPhysSQLiteDriverLink1: TFDPhysSQLiteDriverLink;
51     FDConnection1: TFDConnection;
52     FDQuery1: TFDQuery;
53     Button2: TButton;
54     DataSource1: TDataSource;
55     FDSchemaAdapter1: TFDSchemaAdapter;
56     IMLImages: TImageList;
57     MMUMain: TMainMenu;
58     MNUFile: TMenuItem;
59     MNUPrint: TMenuItem;
60     MNUPreview: TMenuItem;
61     N1: TMenuItem;
62     MNUExit: TMenuItem;
63     IMLMenu: TImageList;
64     MNUImport: TMenuItem;
65     MNUAttendanceData: TMenuItem;
66     MNUFullBackup: TMenuItem;
67     MNUTools: TMenuItem;
68     MNUCreateMainDatabase: TMenuItem;
69     N2: TMenuItem;

```

```

69     MNUBackupandRestore: TMenuItem;
70     MNUBackup: TMenuItem;
71     MNURestore: TMenuItem;
72     PANMenu: TPanel;
73     CMBStandard: TComboBox;
74     BTNSplit1: TButton;
75     Label1: TLabel;
76     Label2: TLabel;
77     CMBSummary: TComboBox;
78     CHKEExcel: TCheckBox;
79     CHKToPDF: TCheckBox;
80     Label4: TLabel;
81     Label3: TLabel;
82     LBToPDF: TLabel;
83     StatusBar1: TStatusBar;
84     SpeedButton2: TSpeedButton;
85     SpeedButton1: TSpeedButton;
86     LBLQueryResults: TLabel;
87     Button3: TButton;
88     FRReports1: TFRReports;
89     procedure Button1Click(Sender: TObject);
90     procedure Button2Click(Sender: TObject);
91     procedure FormCreate(Sender: TObject);
92     procedure MNUExitClick(Sender: TObject);
93     procedure MNUCreateMainDatabaseClick(Sender: TObject);
94     procedure CHKToPDFClick(Sender: TObject);
95     procedure CHKEExcelClick(Sender: TObject);
96     procedure CMBStandardDrawItem(Control: TWinControl; Index: Integer;
97         Rect: TRect; State: TOwnerDrawState);
98     procedure CMBSummaryDrawItem(Control: TWinControl; Index: Integer;
99         Rect: TRect; State: TOwnerDrawState);
100    procedure Button3Click(Sender: TObject);
101    procedure FRReports1Button1Click(Sender: TObject);
102    //report
103    //    procedure Button1Click(Sender: TObject);
104    //    procedure BTNCloseClick(Sender: TObject);
105    //custom report events
106    procedure ReportPaint(Sender: TObject);
107    procedure ButtonPrevClick(Sender: TObject);
108    procedure ButtonNextClick(Sender: TObject);
109    procedure ButtonPrintClick(Sender: TObject);
110    //    procedure Button2Click(Sender: TObject);
111    //    procedure Button3Click(Sender: TObject);
112
113    private
114        { Private declarations }
115        const CNST_STR_DATABASE_LOCATION = 'C:\SafeScanData\Database\';
116        const CNST_STR_COPIEDDATAFILES_LOCATION = 'C:\SafeScanData\CopiedDataFiles';
117        const CNST_STR_ATTENDANCELOGFILENAME = '1_attlog.dat';
118        const CNST_STR_TAFULLBACKUPFILENAME = 'ZKDB.db';
119        const CNST_STR_SQLITEDATABASEFILENAME = 'C:\SafeScanData\Database\main.db';
120        const CNST_STR_BACKUP_LOCATION = 'C:\SafeScanData\backups';
121    public
122        { Public declarations }
123    end;
124
125    var
126        FRMMain: TFRMMain;
127        CANCanvas :TCanvas;
128
129        //custom report vars
130        frmTemp      :TForm;
131        BTNPrint     :TButton;
132        BTNPrev      :TButton;
133        BTNNext      :TButton;
134
135        //form vars
136        strDataIn :TStringList;
137        intPage, intRecsRead, intTotalRecs, intNumPages, intYPOS, intLastReadLine,

```

```

138     intExtraLines : Integer;
139     dblNumPagesCalc: Double;
140     blnHeader: Boolean;
141
142     const CNST_INT_MARGIN_LEFT = 20;
143     const CNST_INT_MARGIN_TOP = 20;
144     const CNST_INT_MARGIN_BOTTOM = 20;
145     const CNST_INT_MARGIN_RIGHT = 20;
146     const CNST_INT_HEADERSIZE = 80;
147     const CNST_INT_PAGENUMX = 620;
148     const CNST_INT_PAGENUMY = 620;
149     const CNST_INT_MAXDETAILLINES = 26;
150     const CNST_STR_BLANKLINE =
151         '
152         ';
153
154 implementation
155
156 {$R *.dfm}
157
158 procedure TFRMMain.Button1Click(Sender: TObject);
159 begin
160     self.FDConnection1.Params.Add('DriverID=SQLite');
161     self.FDConnection1.Params.Add('Database=' + CNST_STR_COPIEDDATAFILES_LOCATION +
162     CNST_STR_TAFULLBACKUPFILENAME);
163     self.FDConnection1.Open;
164     self.FDQuery1.Open;
165
166 end;
167
168 procedure TFRMMain.Button2Click(Sender: TObject);
169 begin
170     self.FDConnection1.Close;
171     self.Close;
172
173 end;
174
175 procedure TFRMMain.Button3Click(Sender: TObject);
176 begin
177     self.FRAREports1.Button1.Click();
178
179 end;
180
181 procedure TFRMMain.CHKExcelClick(Sender: TObject);
182 begin
183     if self.CHKExcel.Checked then
184         self.CHKToPDF.Checked:=false
185     else
186         self.CHKToPDF.Checked :=true;
187
188 end;
189
190 procedure TFRMMain.CHKToPDFClick(Sender: TObject);
191 begin
192     if self.CHKToPDF.Checked then
193         self.CHKExcel.Checked:=false
194     else
195         self.CHKExcel.Checked :=true;
196
197 end;
198
199 procedure TFRMMain.CMBStandardDrawItem(Control: TWinControl; Index: Integer;
200     Rect: TRect; State: TOwnerDrawState);
201 begin
202     self.CMBStandard.Canvas.FillRect(Rect);
203     self.CMBStandard.Canvas.TextOut(Rect.Left,Rect.Top,self.CMBStandard.Items[Index]);
204
205 end;
206
207 procedure TFRMMain.CMBSummaryDrawItem(Control: TWinControl; Index: Integer;
208     Rect: TRect; State: TOwnerDrawState);
209 begin

```

```

203 self.CMBSummary.Canvas.FillRect(Rect);
204 self.CMBSummary.Canvas.TextOut(Rect.Left,Rect.Top,self.CMBSummary.Items[Index]);
205 end;
206
207 procedure TFRMMain.FormCreate(Sender: TObject);
208 begin
209 self.PANMenu.Color:= TColor(RGB(0,48,88));
210 self.CMBStandard.Color:= TColor(RGB(0,88,122));
211 self.CMBSummary.Color:= TColor(RGB(0,88,122));
212 self.Color:= TColor(RGB(0,88,122));
213 end;
214
215 procedure TFRMMain.FRAReports1Button1Click(Sender: TObject);
216 begin
217     FRAReports1.Button1Click(Sender);
218
219 end;
220
221 procedure TFRMMain.MNUCreateMainDatabaseClick(Sender: TObject);
222 {
223     Created 06/10/2023 By Roger Williams
224
225     Creates main.db using SQLite as this is the same as the clocking machines use
226     Creates the tables from the text file:
227
228     schema.txt
229
230     Why? - means easy to change the database without having to recompile the
231           main program, handy during development :)
232 }
233
234 procedure CreateTables;
235 var
236     tmpField : TField;
237
238 begin
239
240     self.FDQuery1.SQL.Text :=
241         'CREATE TABLE IF NOT EXISTS MyTable ('+
242         'ID LARGEINT PRIMARY KEY,'+
243         'Name NVARCHAR(80) NOT NULL'+
244         ');'; //don't use "WITHOUT ROWID" here
245     self.FDQuery1.ExecSQL;
246
247 end;
248
249 begin
250
251 if not FileExists(CNST_STR_COPIEDDATAFILES_LOCATION + CNST_STR_SQLITEDATABASEFILENAME)
252 then
253     begin
254         self.FDConnection1.Params.Add('DriverID=SQLite');
255         self.FDConnection1.Params.Add('Database=' + CNST_STR_COPIEDDATAFILES_LOCATION +
256             CNST_STR_SQLITEDATABASEFILENAME);
257         self.FDConnection1.Open;
258         CreateTables;
259     end
260 else
261     showmessage('File Already Exists - Nothing To Be Done!');
262
263 end;
264
265 procedure TFRMMain.MNUExitClick(Sender: TObject);
266 begin
267     self.Close;
268 end;
269

```

```

270 {
271     //adds record to table
272     var
273         tmpTable : TFDTable;
274
275     tmpTable.Connection := self.FDConnection1;
276     tmpTable.Fields.Clear;
277     tmpTable.TableName := 'MyTable';
278     tmpTable.Open();
279     tmpTable.InsertRecord([1, 'First']);
280     FDConnection1.Commit;
281 }
282
283
284
285     //report
286
287 {
288 procedure TFRMMain.BTNCloseClick(Sender: TObject);
289 begin
290     frmTemp.Close;
291     frmTemp.DisposeOf;
292 end;
293
294 procedure TFRMMain.Button1Click(Sender: TObject);
295 var
296     strTemp, strTemp2:string;
297     intNum:integer;
298
299 begin
300     //self.Print //creates BMP of form which is then sent to the printer!
301
302     //read file
303     strDataIn:=TStringList.Create;
304     strDataIn.LoadFromFile('C:\Projects\Delphi\SafeScanSuite\DBreader and
305 Import\Database\schemas.txt');
306     blnHeader:=true;
307     intPage:=1;
308     intRecsRead:=0;
309     //get number of lines read
310     intTotalRecs:=strDataIn.Count;
311     //calculate how many pages using above plus 10 for header/footer
312
313     if intTotalRecs <= CNST_INT_MAXDETAILLINES then
314         intNumPages:=1
315     else
316     begin
317         dblNumPagesCalc:= intTotalRecs / CNST_INT_MAXDETAILLINES; // (CNST_INT_MAXDETAILLINES
318         +10);
319         strTemp:=dblNumPagesCalc.ToString;
320
321         if strTemp.IndexOf('.') <> 0 then
322         begin
323             strTemp2:='    ';
324             strTemp.CopyTo(0,strTemp2[strTemp.IndexOf('.')],0,strTemp.IndexOf('.'));
325             strTemp2:=strTemp2.Trim;
326             intNumPages:=StrToInt(strTemp2)+1;
327             intExtraLines:= intTotalRecs mod CNST_INT_MAXDETAILLINES;
328         end;
329     end;
330
331     //configure "report" screen
332     frmTemp:=TForm.Create(self);
333     frmTemp.Tag:=1;
334     frmTemp.Color:=clWhite;
335     frmTemp.Top:=0;
336     frmTemp.Left:=0;
337     frmTemp.Width:=800;
338     frmTemp.Height:=700;

```

```
337 frmTemp.Position:=poDesigned;
338 frmTemp.OnPaint:=ReportPaint;
339
340 //create buttons
341 BTNPrev:=TButton.Create(frmTemp);
342 BTNNext:=TButton.Create(frmTemp);
343 BTNPrint:=TButton.Create(frmTemp);
344
345 BTNPrint.Height:=20;
346 BTNNext.Height:=17;
347 BTNPrev.Height:=17;
348
349 BTNPrint.Width:=60;
350 BTNNext.Width :=40;
351 BTNprev.Width :=80;
352
353 BTNPrint.Top:=638;
354 BTNPrev.Top:=642;
355 BTNNext.Top:=642;
356
357 BTNPrev.Left:=30;
358 BTNPrint.Left:=342;
359 BTNNext.Left:=550;
360
361 BTNPrev.Name:='BTNPrev';
362 BTNNext.Name:='BTNNext';
363 BTNPrint.Name:='BTNPrint';
364
365 BTNPrev.Caption:='Previous';
366 BTNNext.Caption:='Next';
367 BTNPrint.Caption:='Print';
368
369 BTNPrev.OnClick:=ButtonPrevClick;
370 BTNNext.OnClick:=ButtonNextClick;
371 BTNPrint.OnClick:=ButtonPrintClick;
372
373 BTNPrev.Parent:=frmTemp;
374 BTNNext.Parent:=frmTemp;
375 BTNPrint.Parent:=frmTemp;
376
377 BTNNext.Tag:=1;
378
379 //show form and populate
380 frmTemp.Visible:=True;
381 BTNNext.SetFocus;
382 end;
383
384 procedure TFRMMain.Button2Click(Sender: TObject);
385 begin
386 //populate "view" form
387 frmTemp.Tag:=1;
388 ReportPaint(Sender);
389 end;
390
391 procedure TFRMMain.Button3Click(Sender: TObject);
392 begin
393 if self.Button3.Caption = 'Hide Report Form' then
394 begin
395 frmTemp.Visible:=False;
396 self.Button3.Caption:='Show Report Form';
397 end
398 else
399 begin
400 frmTemp.Visible:=True;
401 self.Button3.Caption:='Hide Report Form';
402 end;
403
404 end;
405 }
```

```

406
407
408 //*****report custom events*****
409 procedure TFRMMain.ButtonPrintClick(Sender: TObject);
410 begin
411   BTNPrint.Tag:=1;
412 end;
413
414 procedure TFRMMain.ButtonNextClick(Sender: TObject);
415 begin
416   intPage:=intPage+1;
417
418   if IntPage > intNumPages then
419     intPage:=intNumPages;
420
421   BTNPrev.Tag:=0;
422   BTNNext.Tag:=1;
423   frmTemp.Tag:=1;
424   ReportPaint(Sender);
425 end;
426
427 procedure TFRMMain.ButtonPrevClick(Sender: TObject);
428 begin
429   if intPage = IntNumPages then
430     intLastReadLine:= strDataIn.Count-(CNST_INT_MAXDETAILLINES+intExtraLines);
431
432   if (intPage < IntNumPages) and (IntPage >1) then
433     intLastReadLine:= intLastReadLine-CNST_INT_MAXDETAILLINES;
434
435   intPage:=intPage-1;
436
437   if IntPage <1 then
438     intPage:=1;
439
440   BTNPrev.Tag:=1;
441   BTNNext.Tag:=0;
442   frmTemp.Tag:=1;
443   ReportPaint(Sender);
444 end;
445
446 // Use this to capture a rectangle on the screen...
447 function CaptureScreenRect( ARect: TRect ): TBitmap;
448 var
449   ScreenDC: HDC;
450 begin
451   Result := TBitmap.Create;
452   with Result, ARect do
453   begin
454     Width := Right - Left;
455     Height := Bottom - Top;
456     ScreenDC := GetDC( 0 );
457     try
458       BitBlt( Canvas.Handle, 0, 0, Width, Height, ScreenDC,
459         Left, Top, SRCCOPY );
460     finally
461       ReleaseDC( 0, ScreenDC );
462     end;
463   end;
464 end;
465
466 { Use this to capture an entire form or control... }
467 function CaptureControlImage( Control: TControl ): TBitmap;
468 begin
469   with Control do
470   if Parent = nil then
471     Result := CaptureScreenRect( Bounds( Left, Top, Width,Height ))
472   else
473     with Parent.ClientToScreen( Point( Left, Top )) do
474       Result := CaptureScreenRect( Bounds( X, Y, Width,

```

```

475 Height));
476 end;
477
478 procedure TFRMMain.ReportPaint(Sender: TObject);
479 var
480     intNum      :integer;
481     bmp:TBitmap;
482
483 begin
484
485     if BTNPrint.tag = 1 then
486     begin
487         BTNPrint.tag:=0;
488         bmp:=Tbitmap.Create;
489         bmp:=CaptureControlImage(frmTemp);
490         bmp.SaveToFile('C:\temp\bmpptest.bmp');
491     end;
492
493
494     if frmTemp.Tag <> 0 then
495     begin
496         //create report header
497         frmTemp.Canvas.Font.Size:=10;
498         frmTemp.Canvas.Font.Name:='Tahoma';
499         frmTemp.Canvas.Font.Color:=clBlack;
500
501         frmTemp.Canvas.Font.Size:=14;
502         frmTemp.Canvas.Font.Style:=[fsBold , fsUnderline];
503         frmTemp.Canvas.TextOut(CNST_INT_MARGIN_LEFT,CNST_INT_MARGIN_TOP,'Report Header');
504         frmTemp.Canvas.Font.Size:=10;
505         frmTemp.Canvas.TextOut(620,CNST_INT_MARGIN_TOP,'Page: ' + intPage.ToString + ' ' +
DateToStr(Date));
506         frmTemp.Canvas.MoveTo(CNST_INT_MARGIN_LEFT,60);
507         frmTemp.Canvas.LineTo(760,60);
508
509         //clear page
510         for intNum := 0 to CNST_INT_MAXDETAILLINES-1 do
511         begin
512             frmTemp.Canvas.Font.Size:=8;
513             frmTemp.Canvas.Font.Style:=[];
514             frmTemp.Canvas.TextOut(CNST_INT_MARGIN_LEFT,CNST_INT_HEADERSIZE+(intNum*20),
CNST_STR_BLANKLINE);
515         end;
516
517         frmTemp.Canvas.Font.Style:=[];
518         intYPOS:=0;
519
520         //write detail lines
521         if intPage = 1 then
522         begin
523             for intNum := 0 to CNST_INT_MAXDETAILLINES-1 do
524             begin
525                 frmTemp.Canvas.Font.Size:=8;
526                 frmTemp.Canvas.TextOut(CNST_INT_MARGIN_LEFT,CNST_INT_HEADERSIZE+(intNum*20),
strDataIn[intNum]);
527             end;
528         end
529         else
530         begin
531             if intLastReadLine = strDataIn.Count then
532                 if intExtraLines <> 0 then
533                     intLastReadLine := strDataIn.Count-intExtraLines
534                 else
535                     intLastReadLine := strDataIn.Count;
536
537             for intNum := intLastReadLine+1 to intLastReadLine+CNST_INT_MAXDETAILLINES-1 do
538             begin
539                 if intNum <= strDataIn.Count-1 then
540                     begin

```

```
541         frmTemp.Canvas.Font.Size:=8;
542         frmTemp.Canvas.TextOut (CNST_INT_MARGIN_LEFT,CNST_INT_HEADERSIZE+(intYPos*20),
          strDataIn[intNum]);
543         intYPos:=intYPos+1;
544     end;
545 end;
546 end;
547
548 if BTNNext.Tag = 1 then
549     if intPage = intNumPages then
550         intLastReadLine:=strDataIn.Count
551     else
552         intLastReadLine:=intNum;
553
554
555 //print footer
556 frmTemp.Canvas.TextOut (CNST_INT_PAGENUMX,CNST_INT_PAGENUMY,'Page: ' + intPage.ToString +
    ' ' + DateToStr (Date));
557 frmTemp.Tag:=0;
558 end;
559 end;
560
561 end.
562
```